



Universidade do Minho

Programação Orientada aos Objetos

Licenciatura em Ciências da Computação

2025/2026

Grupo 42



Moisés Ferreira
A97020



Rita Machado
A102508



Oliver Teixeira
A102506

Conteúdo

1. Introdução.....	3
2. Classes.....	4
2.1 Dispositivo	4
2.2 ColunaSom	4
2.3 Cortina	5
2.4 Lampada	5
2.5 PortaoGaragem	6
2.6 Rele	6
2.7 Divisão	7
2.8 Casa	7
2.9 Utilizador	8
2.10 TipoUtilizador	8
2.11 Admin	9
2.12 Normal	9
2.13 DomusControl	10
2.14 DomusControlManager	10
2.15 InfoDivisao	11
2.16 Main	11
3. Estrutura do projeto	12
4. Lógica de Simulação de tempo e Cálculo de consumos.....	13
5. Funcionalidades da aplicação	14
6. Diagrama de classes	19
7. Conclusão	20

1. Introdução

Este projeto consiste na construção e implementação do DomusControl, uma plataforma interativa de simulação e gestão de dispositivos inteligentes numa casa.

A nível técnico, o desenvolvimento deste projeto foi realizado em Java e serviu como caso de estudo prático para a aplicação dos pilares da Programação Orientada aos Objetos, nomeadamente Abstração, Herança, Polimorfismo e Encapsulamento.

2. Classes

2.1 Dispositivo

```
public abstract class Dispositivo implements Serializable {  
  
    private String id;  
    private String marca;  
    private String modelo;  
    private double consumoHora;  
    private Estado estado;  
    private double energia;  
    private double tempoUso;  
    private int numeroAtivacoes;  
  
    public enum Estado {  
        LIGADO,  
        DESLIGADO  
    }  
}
```

A nossa classe Dispositivo atua como a superclasse de todos os equipamentos da nossa casa inteligente. Esta classe, definida como abstract, atua como um molde genérico, onde as subclasses concretas herdam os seus atributos, evitando a duplicação de código.

2.2 ColunaSom

```
public class ColunaSom extends Dispositivo implements Serializable {  
  
    private static final long serialVersionUID = 1L;  
    private int volume;  
  
    public ColunaSom(String id, String marca, String modelo, double consumoHora, int volume) {  
        super(id, marca, modelo, consumoHora);  
        this.volume = volume;  
    }  
}
```

A ColunaSom é uma subclasse de Dispositivo, dedicada a um equipamento de áudio, introduzindo a variável de volume. Esta subclasse personaliza o método de execução de comandos para permitir aumentar ou diminuir a intensidade sonora.

2.3 Cortina

```
public class Cortina extends Dispositivo implements Serializable {  
  
    private static final long serialVersionUID = 1L;  
    private int grauAbertura; //0-100%  
    private String cor;  
  
    public Cortina(String id, String marca, String modelo, double consumoHora, int grauAbertura, String cor) {  
        super(id, marca, modelo, consumoHora);  
        this.grauAbertura = grauAbertura;  
        this.cor = cor;  
    }  
}
```

A Cortina é uma subclasse de Dispositivo à qual atribuímos uma cor física e um grau de abertura (de 0 a 100%). O seu comportamento foi adaptado para que o estado do dispositivo passe automaticamente a “LIGADO” (consumindo energia do motor elétrico) sempre que a abertura for superior a 0%.

2.4 Lampada

```
public class Lampada extends Dispositivo implements Serializable {  
  
    private static final long serialVersionUID = 1L;  
  
    private int corLuz; //vai de 2700K ate 4000K  
    private int intensidadeL;  
  
    public Lampada(String id, String marca, String modelo, double consumoHora, int corLuz, int intensidadeL) {  
        super(id, marca, modelo, consumoHora);  
        this.corLuz = corLuz;  
        this.intensidadeL = intensidadeL;  
    }  
}
```

A Lampada é uma subclasse da classe Dispositivo onde acrescentamos atributos próprios de iluminação, nomeadamente a cor da luz (tonalidade) e a intensidade luminosa. A classe reescreve a lógica de comandos para conseguir interpretar e ajustar esses valores específicos.

2.5 PortaoGaragem

```
public class PortaoGaragem extends Dispositivo implements Serializable {  
  
    private static final long serialVersionUID = 1L;  
    private int grauAbertura; //0-100%  
  
    public PortaoGaragem(String id, String marca, String modelo, double consumoHora, int grauAbertura) {  
        super(id, marca, modelo, consumoHora);  
        this.grauAbertura = grauAbertura;  
    }  
}
```

O PortaoGaragem estende a classe Dispositivo para simular o controlo de acessos motorizados. À semelhança da Cortina, possui um grau de abertura ajustável, ativando o seu estado de consumo de energia sempre que recebe ordem para abrir ou fechar.

2.6 Rele

```
private static final long serialVersionUID = 1L;  
  
public Rele(String id, String marca, String modelo, double consumoHora) {  
    super(id, marca, modelo, consumoHora);  
}
```

O Rele é uma subclasse mais direta e simples do Dispositivo. Atua como um interruptor inteligente básico, sem variáveis adicionais, servindo puramente para ligar ou desligar e simular o consumo de aparelhos elétricos tradicionais que estejam conectados ao sistema.

2.7 Divisão

```
public class Divisao implements Serializable {  
    private String nome;  
    private Map<String, Dispositivo> dispositivos;
```

A Divisão representa um compartimento específico da casa e gere o conjunto de dispositivos nele instalados através de um Map. Esta classe assegura o encapsulamento ao utilizar clones nas operações de adição e consulta, servindo ainda de ponte para a execução de comandos e para a propagação da simulação do tempo a todos os dispositivos nela contidos.

2.8 Casa

```
public class Casa implements Serializable {  
    private String morada;  
    private String dono;  
    private Map<String, Divisao> divisoes;
```

A Casa é uma classe responsável por agrupar as várias divisões de uma residência através de um Map. Gere a morada e a identificação do dono, servindo como o ponto de entrada para a gestão de dispositivos e para a propagação da simulação do tempo a todas as divisões. Garantimos o encapsulamento e a integridade dos dados com a utilização de cópias (clone) em todas as operações de consulta e modificação.

2.9 Utilizador

```
public class Utilizador implements Serializable {  
    private String nome;  
    private Map<String, TipoUtilizador> casasAcesso;
```

O Utilizador é a classe que identifica os indivíduos no sistema através de um nome. Utiliza um Map para gerir as permissões de acesso, associando cada morada de casa a um nível de autoridade específico (TipoUtilizador). Essa estrutura permite se o sistema valide se o utilizador tem permissão para realizar operações administrativas ou apenas consultas, garantindo a segurança através do uso de cópias.

2.10 TipoUtilizador

```
public interface TipoUtilizador extends Serializable {  
    String getDesignacao();  
    TipoUtilizador clone();
```

O TipoUtilizador é uma interface que atua como um contrato para o sistema de permissões. Não contém lógica própria, servindo apenas para garantir que qualquer nível de acesso implementado saiba fornecer a sua designação e criar uma cópia de si mesmo.

O Admin e Normal são classes concretas que implementam a interface TipoUtilizador.

2.11 Admin

```
public class Admin implements TipoUtilizador {  
  
    //construtor por omissão  
    public Admin() {}  
  
    @Override  
    public String getDesignacao() {  
        return "Administrador";  
    }  
}
```

O Admin identifica os utilizadores com permissões totais sobre uma residência (como adicionar/remover divisões ou dispositivos).

2.12 Normal

```
public class Normal implements TipoUtilizador {  
  
    public Normal() {}  
  
    @Override  
    public String getDesignacao() {  
        return "Utilizador Normal";  
    }  
}
```

Já a classe Normal, restringe o acesso apenas para a interação básica.

2.13 DomusControl

```
public class DomusControl implements Serializable {  
  
    private Map<String, Casa> casas;  
    private Map<String, Utilizador> utilizadores;
```

O DomusControl é o núcleo central do sistema, funcionando como a base de dados em memória que agrega todos os utilizadores e casas através de estruturas Map.

Esta é a classe responsável pela implementação dos métodos de leitura e escrita em ficheiros binários. Além disso, atua como o coordenador principal da simulação do avanço do tempo e as coordenadas de comando de toda a hierarquia de objetos.

2.14 DomusControlManager

```
public class DomusControlManager implements Serializable {  
  
    private DomusControl sistema;  
    private Scanner scanner;  
    private String utilizadorAtivo;
```

O DomusControlManager é a classe responsável pela interface entre o utilizador e o sistema, gerindo todos os menus de navegação e a entrada de dados. Atua como coordenador da interação com o DomusControl, sendo responsável por gerir a sessão do utilizador ativo, validar inputs e apresentar o resultado dos comandos.

2.15 InfoDivisao

```
public class InfoDivisao {  
  
    private String morada;  
    private String divisao;  
    private int quantidade;  
}
```

Esta classe atua como classe auxiliar de transporte de dados. Serve para agregar e encapsular informações pontuais e estatísticas, agregando por exemplo, a morada da casa, o nome da divisão e uma quantidade específica. Esta estrutura facilita a geração de relatórios e a transferência de dados para os menus do sistema, sem expor ou sobrecarregar a lógica das classes principais.

2.16 Main

```
public class Main {  
  
    Run | Debug | Run main | Debug main  
    public static void main(String[] args) throws FileNotFoundException, IOException, ClassNotFoundException, Exception {  
  
        DomusControlManager domus = new DomusControlManager();  
  
        domus.run(args);  
    }  
}
```

A Main é a classe que serve como ponto de entrada da aplicação. Sua única responsabilidade é instanciar o DomusControlManager e iniciar a execução do programa, delegando ao gestor toda a lógica de interação e controlo do sistema.

3. Estrutura do projeto

Nosso projeto está organizado em três camadas:

A camada de dados (o modelo): é composta pelas classes que representam o negócio: Dispositivo (e suas subclasses Lampada, Cortina, ColunaSom, PortaoGaragem, Rele), Divisao, Casa, Utilizador (e suas permissões TipoUtilizador, Admin, Normal), a classe auxiliar InfoDivisao e a base de dados central DomusControl.

A camada de interação com o utilizador: esta é composta por métodos de formatação de output (como o toString()) que apresentam o estado do sistema na consola, sendo geridos dentro do DomusControlManager.

A camada de controlo do fluxo do programa: é composta pela classe Main (que inicia a aplicação) e pela classe DomusControlManager que atua também como orquestrador ao receber os inputs do utilizador, validar dados e invocar as alterações no modelo.

4. Lógica de Simulação de tempo e Cálculo de consumos

Com a funcionalidade de simulação temporal, permitimos calcular o impacto energético do uso da nossa casa inteligente. Esta lógica foi implementada recorrendo a um efeito cascata que percorre a hierarquia do sistema, garantindo que cada classe tenha uma responsabilidade bem definida:

Iniciação do controlador: O utilizador insere no DomusControlManager o número de horas que deseja avançar no tempo. O gestor envia esse valor diretamente para o núcleo do sistema (DomusControl).

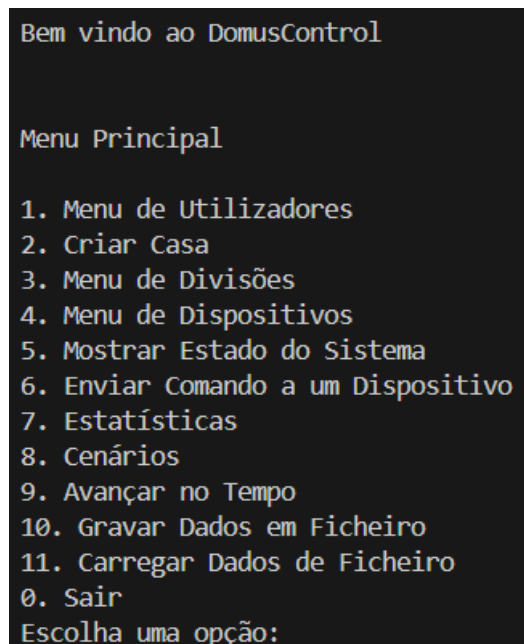
Propagação Hierárquica: O DomusControl percorre o mapa de todas as casas registadas e chama o método `simularTempo(horas)`. Cada casa faz o mesmo para as suas divisões e cada Divisao faz o mesmo para todos os aparelhos na sua lista.

Cálculo no Dispositivo: A responsabilidade final do cálculo recai sobre a classe Dispositivo. Esta, quando recebe o comando do tempo, o aparelho verifica o seu Estado. Se tiver LIGADO, multiplicamos o seu consumoHora pelas horas decorridas e adiciona esse valor à sua variável de energia total. Se tiver DESLIGADO, não acumula qualquer valor.

5. Funcionalidades da Aplicação

Neste capítulo do relatório, serão apresentadas as principais funcionalidades do DomusControl.

Navegação do Menu Principal: A aplicação inicia-se com um menu principal intuitivo em que mostra todas as opções do sistema. A partir deste menu, o utilizador pode aceder à gestão de contas, administração das casas, controlo de dispositivos, ver estatísticas e opções de dados como carregar ou guardar os dados em um ficheiro.

A screenshot of a terminal window showing the DomusControl application's main menu. The text is as follows:

```
Bem vindo ao DomusControl

Menu Principal

1. Menu de Utilizadores
2. Criar Casa
3. Menu de Divisões
4. Menu de Dispositivos
5. Mostrar Estado do Sistema
6. Enviar Comando a um Dispositivo
7. Estatísticas
8. Cenários
9. Avançar no Tempo
10. Gravar Dados em Ficheiro
11. Carregar Dados de Ficheiro
0. Sair
Escolha uma opção:
```

Gestão de Utilizadores e Controlo de Acesso: O nosso sistema suporta a criação de múltiplos utilizadores e implementa um sistema de sessões (Login). Dependendo da casa em que estamos a aceder, o utilizador ativo pode ter permissões de Administrador (podendo alterar a estrutura da casa) ou de utilizador Normal (ficando limitado somente à interação básica com os dispositivos).

```
Menu de Utilizadores

Utilizador atual: Nenhum
1. Registrar Novo Utilizador
2. Efetuar Login
3. Ver Lista de Utilizadores
4. Remover Utilizador
5. Definir Permissões
0. Voltar ao Menu Principal
Escolha uma opção: |
```

Criação de Casas e Divisões: Através das opções de administrador, utilizadores com esses privilégios podem registar novas casas e estruturá-las criando também quantas divisões quiser (por exemplo sala, quarto, garagem, etc).

```
Insira a morada da casa:
rua a
Insira o nome do dono da casa:
utilizador1
Casa criada! Morada: rua a, Dono: utilizador1

Menu de Divisões

1. Adicionar Nova Divisão
2. Remover Divisão Existente
0. Voltar ao Menu Principal
Escolha uma opção: |
```

Criação de Dispositivos: Uma das funcionalidades do nosso sistema é a capacidade de adicionar novos dispositivos às divisões. O utilizador pode indicar o tipo de dispositivo (lâmpada, cortina, rele...) e o sistema instância a classe automaticamente e em seguida pede alguns parâmetros específicos como (como o ID e o consumo energético).

```
MENU DE DISPOSITIVOS

1. Adicionar Novo Dispositivo
2. Remover Dispositivo Existente
0. Voltar ao Menu Principal
Escolha uma opção: |
```

Interações com os Dispositivos: O utilizador pode enviar comandos para os dispositivos de uma casa. Além dos comandos básicos como “LIGAR” e “DESLIGAR”, o sistema também suporta o envio de comandos específicos com por exemplo alterar a “INTENSIDADE” de uma lâmpada para 80% ou alterar o “GRAU” de abertura de um portão, demonstrando o funcionamento do polimorfismo.

```
Comandos a Dispositivos

  Enviar Comando
Insira a morada da casa:
rua a
Insira o nome da divisão:
sala
Insira o ID do dispositivo:
l1
Insira o comando:
LIGAR
Comando executado com sucesso!
```

Visualização do estado da casa: No menu, também é possível com que o utilizador possa ver o estado atual da casa.

```
LISTA DE CASAS
[Casa] Morada: rua a | Dono: utilizador1
-> Divisao: sala
    - [Lampada] ID: l1 | Estado: LIGADO | Consumo por hora: 10.0 Wh | Energia Consumida: 0.0 Wh | Cor: 2700K | Intensidade: 0%
```

Simulação do Tempo e Cálculo do Consumo: O nosso sistema também possui a funcionalidade de “avançar no tempo”, onde o utilizador determina quantas horas avançar. Ao fazê-lo, o sistema percorre todos os dispositivos e atualiza a energia consumida por cada dispositivo que se encontra no estado “LIGADO”, permitindo consultar o consumo total da casa.


```
SIMULAÇÃO DE TEMPO
Quantas horas deseja avançar no sistema?
Horas: 10

O tempo avançou 10.0 horas em todas as casas!
Os consumos foram atualizados com base nos dispositivos que estavam LIGADOS.
```

Carregar e Guardar dados: Para garantir que o utilizador não perca todos os dados depois da utilização do DomusControl, o sistema oferece opções para exportar todos os dados para um ficheiro binário local e para recarregar estes dados numa futura utilização da aplicação.

```
10. Gravar Dados em Ficheiro
11. Carregar Dados de Ficheiro
```

Estatísticas de Utilização: Para além do cálculo do consumo energético, o sistema também verifica a utilização dos equipamentos. O sistema regista o tempo total de uso (em horas) e o número de ativações de cada aparelho. Esta implementação permite o sistema gerar relatórios estatísticos como por exemplo apresentar “Top 3 de Dispositivos mais usados” de uma casa.

```
MENU DE ESTATISTICAS

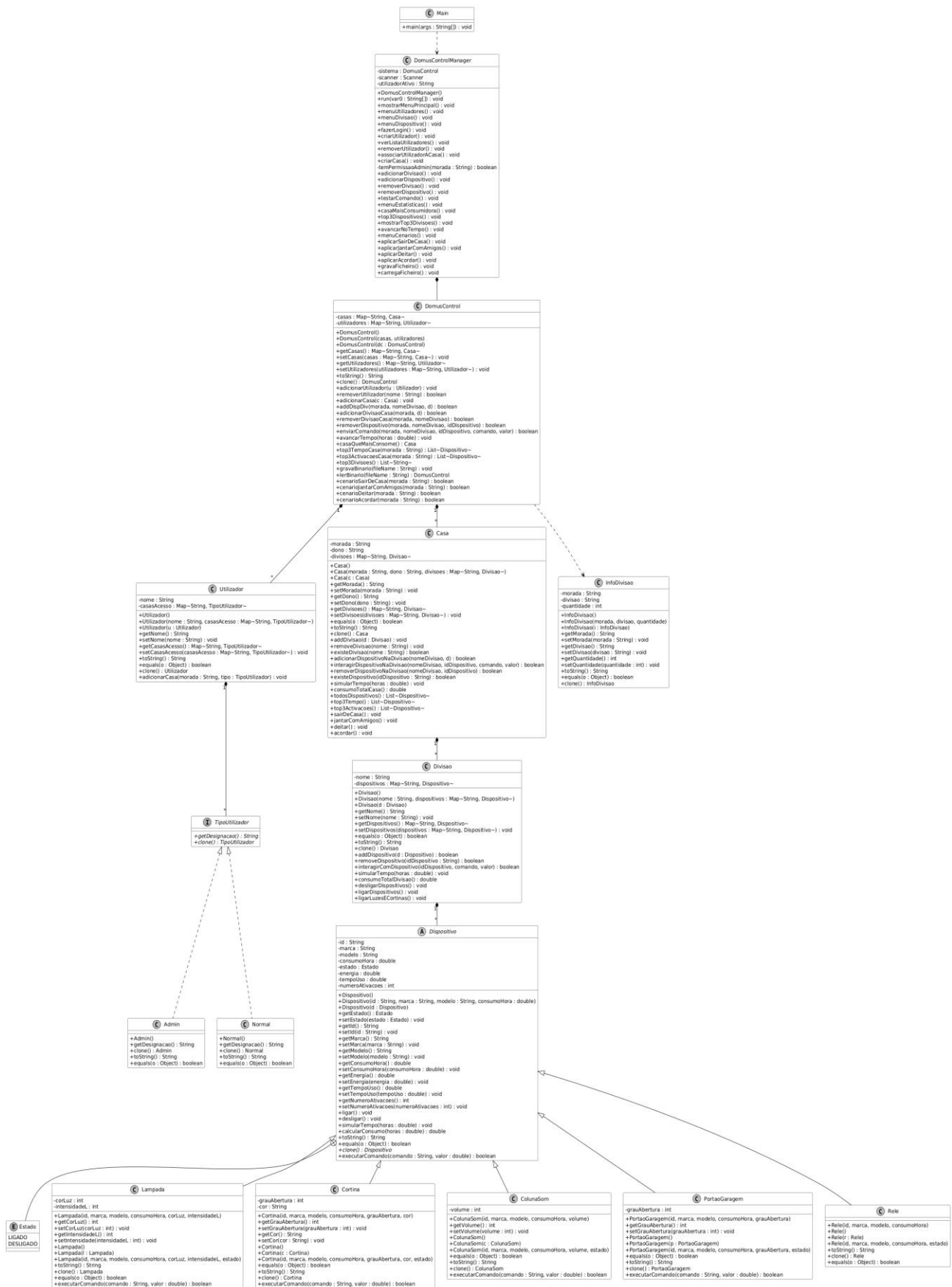
1. Casa que mais consome
2. Top 3 dispositivos mais usados numa casa
3. Top 3 divisões com mais dispositivos
0. Voltar ao Menu Principal
Escolha uma opção: █
```

Cenários e Rotinas: O nosso sistema também implementa cenários pré-definidos que mandam comandos para múltiplos dispositivos de diferentes divisões em simultâneo. Através do

menu de Cenários, o utilizador pode invocar rotinas como “Sair de casa” (que percorre toda a árvore de objetos e desliga todos os dispositivos) ou “Acordar” (que identifica automaticamente as divisões do tipo “Quarto”, acendendo as luzes e abrindo as cortinas com grau de 100%).

```
MENU DE CENARIOS  
  
1. Sair de Casa  
2. Jantar com Amigos  
3. Deitar  
4. Acordar  
0. Voltar ao Menu Principal  
Escolha uma opção: █
```

6. Diagrama de Classes



7. Conclusão

A nível geral, com o desenvolvimento do DomusControl, concluímos o nosso desafio. A construção deste projeto revelou-se um excelente exercício prático para a consolidação da linguagem Java.

A nível de arquitetura, o sistema assentou nos pilares da Programação Orientada aos Objetos. Destaca-se a aplicação da Herança e Polimorfismo na hierarquia dos Dispositivos, e o rigoroso encapsulamento dos dados através do uso de cópias (clone).

Em suma, o resultado é um sistema limpo e extensível onde acreditamos ter cumprido com o objetivo proposto.